

✓ GITHUB TEMPLATES & FILE EXAMPLES

Copy-paste ready templates for your project repository

Complete Repository Structure

```
cv-project-inventory/
├── README.md
├── requirements.txt
├── .gitignore
├── LICENSE
├──
├── data/
│   ├── raw/
│   ├── processed/
│   ├── sample/
│   └── README.md
├──
├── models/
│   ├── pretrained/
│   ├── trained/
│   └── README.md
├──
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   ├── 02_model_training.ipynb
│   ├── 03_evaluation.ipynb
│   └── 04_demo.ipynb
├──
├── src/
│   ├── __init__.py
│   ├── data_processing.py
│   ├── model.py
│   ├── train.py
│   └── inference.py
├──
├── results/
│   ├── images/
│   ├── metrics.txt
│   └── visualizations/
```

```
|  
└─ docs/  
    ├── proposal.pdf  
    ├── presentation.pdf  
    └─ AI_usage_log.md
```

1. README.md (Midterm Version)

Warehouse Inventory Detection System

![[Project Banner]] (docs/banner.png)

Team Members

- John Smith - js1234@example.edu (Data Collection & Preprocessing)
- Jane Doe - jd5678@example.edu (Model Training & Optimization)

Project Tier

Tier 1: This project uses YOLOv8 for object detection, which is a well-established approach suitable for beginners. We're focusing on getting solid fundamentals right rather than implementing cutting-edge techniques.

Problem Statement

Warehouse workers spend 2+ hours daily manually counting inventory on shelves, leading to errors and inefficiency. A single warehouse with 20 workers wastes \$50,000+ annually on this task. Our system aims to automate this process using computer vision.

Solution Overview

We're building an automated inventory counting system that takes photos of warehouse shelves and uses YOLOv8 object detection to identify and count products. The system will process images in under 1 second and generate automated count reports.

Technical Approach

- **CV Technique:** Object Detection
- **Model:** YOLOv8n (nano version for speed)
- **Framework:** PyTorch + Ultralytics

- **Why this approach:** YOLO is industry-standard for real-time object detection. The nano version offers good accuracy (85%+) while being fast enough for practical warehouse use (>20 FPS).



Dataset

- **Source:** Custom dataset + SKU-110K public dataset
- **Size:**
 - Training: 800 images
 - Validation: 200 images
 - Test: 100 images
- **Labels:** 5 product categories (boxes, bottles, cans, bags, packages)
- **Collection Method:**
 - 500 images collected from local warehouse (with permission)
 - 600 images from SKU-110K dataset
- **Link:** [SKU-110K Dataset](#)



Success Metrics

- **Primary Metric:** mAP50 (mean Average Precision at 50% IoU)
- **Target:** 85%+ detection accuracy
- **Secondary Metrics:**
 - Inference speed: <0.1 seconds per image (>10 FPS)
 - Counting accuracy: <5% error rate on total counts
 - False positive rate: <10%



Week-by-Week Plan

Week	Tasks	Milestone
Week 10 (Oct 30)	- Collect initial 200 images - Set up Colab environment - Label images using Roboflow	Dataset ready for training
Week 11 (Nov 6)	- Complete data collection (800 images) - Fine-tune YOLOv8n on our dataset - Initial training runs	Model training started
Week 12 (Nov 13)	- Optimize hyperparameters - Reach 80%+ accuracy - Test on validation set	Good accuracy achieved
Week 13 (Nov 20)	- Create demo notebook - Record demo video - Test on real warehouse images	Demo completed

Week	Tasks	Milestone
Week 14 (Nov 27)	- Final testing and bug fixes - Complete documentation - Prepare presentation	Everything polished
Week 15 (Dec 4)	- Final presentation - Live demo	🎉 Project complete!

⚠️ Risks & Mitigation

Risk	Probability	Mitigation
Can't collect enough warehouse images	Medium	Use more of SKU-110K public dataset + augmentation
Model accuracy is too low	Low	Try data augmentation, larger model (YOLOv8s), or adjust confidence
Colab GPU time runs out	Medium	Use Kaggle as backup (30 hrs/week free)
Products look too similar	Medium	Add more diverse training data, focus on shape/size features
Real-time speed not achieved	Low	Use smaller input size (320px) or export to ONNX format



Resources Needed

- **Compute:** Google Colab free tier (sufficient for YOLOv8n training)
- **Cost:** \$0 (all free resources)
- **APIs:** None required
- **Tools:**
 - Ultralytics YOLOv8
 - Roboflow (for labeling)
 - Google Colab
 - OpenCV for preprocessing



AI Usage Log

Will be updated throughout the project. See [docs/AI_usage_Log.md](#) for detailed log.

Summary (as of Oct 20):

- Used ChatGPT to understand YOLO architecture
- Used Claude for debugging data loading issues
- Generated visualization code using GitHub Copilot
- All CV model code written by us



Current Status

- ☒ Repository created
- ☒ Proposal written

- ☒ Dataset source identified
 - ☐ Dataset acquired (in progress - 200/800 images)
 - ☐ Model training started
 - ☐ Demo created
 - ☐ Final presentation ready
-

References

1. Ultralytics YOLOv8: <https://docs.ultralytics.com/>
 2. SKU-110K Dataset: https://github.com/eg4000/SKU110K_CVPR19
 3. Object Detection Tutorial:
https://pytorch.org/tutorials/intermediate/torchvision_tutorial.html
-

License

Academic Use Only - ITAI 1378 Course Project

Acknowledgments

- [Name] for guidance and ...
 - Warehouse manager for allowing data collection
 - Ultralytics for YOLOv8 implementation
 - ChatGPT and Claude for learning assistance
-

2. requirements.txt

```
# Deep Learning Frameworks
torch>=2.0.0
torchvision>=0.15.0
ultralytics>=8.0.0

# Computer Vision
opencv-python>=4.8.0
opencv-contrib-python>=4.8.0
Pillow>=10.0.0

# Data Processing
numpy>=1.24.0
pandas>=2.0.0
scipy>=1.10.0
```

```
# Visualization
matplotlib>=3.7.0
seaborn>=0.12.0
plotly>=5.14.0

# Jupyter
jupyter>=1.0.0
ipykernel>=6.22.0
ipywidgets>=8.0.0

# ML Utilities
scikit-learn>=1.3.0
scikit-image>=0.21.0

# Progress Bars
tqdm>=4.65.0

# Optional: For VLM projects
# transformers>=4.30.0
# accelerate>=0.20.0

# Optional: For deployment
# flask>=2.3.0
# streamlit>=1.24.0
```

3. .gitignore

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
env/
venv/
ENV/
build/
develop-eggs/
dist/
downloads/
eggs/
```

```
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg

# Jupyter Notebook
.ipynb_checkpoints
*.ipynb_checkpoints/

# Data files (too large for git)
data/raw/*.jpg
data/raw/*.png
data/raw/*.mp4
data/processed/*.jpg
data/processed/*.png

# Model files (too large for git)
models/trained/*.pt
models/trained/*.pth
models/trained/*.onnx
models/pretrained/*.pt

# Results
results/temp/
*.log

# OS
.DS_Store
Thumbs.db
.idea/
.vscode/
*.swp
*.sw0

# Keep directory structure
!data/raw/.gitkeep
!data/processed/.gitkeep
```

```
!models/trained/.gitkeep
!models/pretrained/.gitkeep
```

4. data/README.md

Dataset Documentation

Overview

This directory contains all data for the Warehouse Inventory Detection project.

Structure

- `raw/`: Original, unprocessed images
- `processed/`: Preprocessed images ready for training
- `sample/`: Small sample dataset for quick testing (included in repo)

Dataset Details

Source

- **Custom Collection**: 500 images from local warehouse
- **Public Dataset**: 600 images from SKU-110K dataset

Size

- **Total Images**: 1,100
- **Training Set**: 800 images (72.7%)
- **Validation Set**: 200 images (18.2%)
- **Test Set**: 100 images (9.1%)

Classes

1. **boxes**: Cardboard boxes
2. **bottles**: Plastic/glass bottles
3. **cans**: Metal cans
4. **bags**: Plastic/paper bags
5. **packages**: Wrapped packages

Format

- **Images**: JPG, 1280x720 pixels

- **Labels:** YOLO format (txt files)
 - Format: `class_id x_center y_center width height`
 - Normalized coordinates (0-1)

Data Collection Process

1. **Warehouse Photos** (Week 10):

- Taken with iPhone 13
- Various lighting conditions
- Different shelf angles
- 500 images total

2. **Public Dataset** (Week 10):

- Downloaded from SKU-110K
- Filtered for relevant categories
- 600 images selected

3. **Labeling** (Week 10-11):

- Using Roboflow annotation tool
- Double-checked for accuracy
- Exported in YOLO format

Data Augmentation

Applied during training:

- Random horizontal flip (50%)
- Random brightness adjustment ($\pm 30\%$)
- Random rotation (± 10 degrees)
- Random scaling (0.8-1.2x)

Preprocessing

1. Resize to 640x640 (YOLO input size)
2. Normalize to [0, 1]
3. Convert to RGB if needed

How to Download Data

Full dataset is not in GitHub (too large). Download from:

```
# Option 1: From Google Drive (team members only)
# Link: [PRIVATE LINK]

# Option 2: Reconstruct from public sources
# 1. Download SKU-110K dataset
wget https://github.com/eg4000/SKU110K_CVPR19/releases/download/v1/SKU110K_fixed.tar.gz
tar -xzf SKU110K_fixed.tar.gz

# 2. Contact team for custom warehouse images (if permitted)
```

Sample Data

A small sample dataset (20 images) is included in [sample/](#) for quick testing:

```
cd data/sample
ls
# Should see: images/ and labels/ folders
```

Data Statistics

Metric	Value
Total Objects	4,523
Avg Objects/Image	4.1
Min Objects/Image	1
Max Objects/Image	23
Image Resolution	1280x720
Total Size	2.3 GB

Citation

If you use the SKU-110K portion:

```
@inproceedings{goldman2019dense,
  title={Dense Object Detection in Retail Environments},
  author={Goldman, Eran and Herzig, Roei and Eisenschstat, Aviv and Goldberger, Jacob and
  booktitle={CVPR},
  year={2019}
}
```

✓ 5. docs/AI_usage_log.md

AI Usage Log

Overview

This document tracks all significant AI assistant usage throughout the project. We used AI tools to **learn** and **accelerate development**, not to replace learning.

Log Entries

1. Understanding YOLO Architecture

Date: October 18, 2025

Tool: ChatGPT-4

Team Member: John Smith

Question Asked:

Explain how YOLOv8 works, specifically:

1. Why is it called "You Only Look Once"?
2. How does it detect objects in a single pass?
3. What are the advantages over two-stage detectors like Faster R-CNN?

Sample Response:

YOLO divides the image into a grid and predicts bounding boxes and class probabilities directly for each grid cell in one forward pass. Unlike Faster R-CNN which first proposes regions then classifies them (two stages), YOLO does everything simultaneously, making it much faster (real-time capable).

What I Learned:

YOLO achieves speed through single-stage architecture
Trade-off: slightly lower accuracy than two-stage detectors, but much faster
YOLOv8 improvements: anchor-free detection, better feature pyramid
This is why YOLO is ideal for our real-time warehouse application

How I Applied It:

Chose YOLOv8n (nano) for our project based on speed requirements
Understood why >20 FPS is achievable
Made informed decision between accuracy and speed trade-offs
Explained this clearly in our proposal

✓ Debugging Data Loading Error

Date: October 22, 2025

Tool: Claude

Team Member: Jane Doe

Problem Encountered:

```
# This code was failing:
dataset = LoadImagesAndLabels('data/train')
# Error: FileNotFoundError: [Errno 2] No such file or directory: 'data/train/images'

***Error Message***:
FileNotFoundError: [Errno 2] No such file or directory: 'data/train/images.txt'
```

AI Conversation: Me: "Getting FileNotFoundError when loading YOLO dataset. Here's my folder structure..." Claude: "YOLO expects either: 1) images.txt with image paths, or 2) direct path to images folder. Try..." Solution Provided:

```
# Instead of:
dataset = LoadImagesAndLabels('data/train')

# Use:
dataset = LoadImagesAndLabels('data/train/images')
```

What I Learned:

YOLO data loaders expect specific directory structures Understanding the difference between dataset path and images path How to read Ultralytics documentation more carefully Importance of following exact YOLO directory conventions

How I Applied It:

Restructured our data folder to match YOLO expectations Created proper data.yaml configuration file Documented correct structure in data/README.md Now understand data loading pipeline better

3. Creating Visualization Code Date: October 25, 2025 Tool: GitHub Copilot Team Member: John Smith Context: Needed to create visualization showing detection results with bounding boxes and labels.

Prompt Given

```
# Explain how to create a function to visualize YOLO detection results
# Input: image path and results object
# Output: matplotlib figure with bounding boxes and labels
```

```
#Generated Code
def visualize_results(image_path, results):
    # Read image
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Create figure
    fig, ax = plt.subplots(1, figsize=(12, 8))
    ax.imshow(img)

    # Draw bounding boxes
    for box in results[0].boxes:
        # Get coordinates
        x1, y1, x2, y2 = box.xyxy[0]
        # ... (AI generated rest)
    ...

***What I Modified**:
```

- Changed color scheme to match our brand
- Added confidence threshold parameter
- Improved label positioning
- Added legend for classes

```
**What I Learned**:
```

- How to extract coordinates from YOLO results object
- matplotlib rectangle patch usage
- Best practices for visualization code structure

```
**Why This Was Okay**:
```

- Visualization code is not a CV learning objective
- I understood and modified the generated code
- Saved time on boilerplate, focused on CV concepts
- Similar to using a library function

```
...
```

✓ 4. Understanding Data Augmentation

Date: October 28, 2025

Tool: ChatGPT-4

Team Member: Jane Doe

Question Asked:

For object detection, which data augmentations are safe to use and which might break the bounding box annotations? Specifically for YOLOv8 training.

Response Summary: Safe augmentations:

- Brightness/contrast
- Rotation (if <15 degrees, boxes rotate)
- Scaling (boxes scale proportionally)

Unsafe/tricky:

Vertical flip (rare in real data) Large rotations (boxes become inaccurate) Crops (might cut objects in half)

What I Learned:

Augmentations must preserve object-box relationships Some augmentations need careful implementation YOLOv8 has built-in safe augmentations Importance of thinking about real-world conditions

How I Applied It:

Configured YOLOv8 augmentation parameters appropriately Avoided aggressive augmentations that might harm training Added augmentation settings to training documentation Understood why validation accuracy improved

5. Code Optimization

Date: November 2, 2025

Tool: ChatGPT-4

Team Member: John Smith

Original Code (slow):

```
# Processing each image individually
for img_path in image_paths:
    img = cv2.imread(img_path)
    result = model(img)
```

```
results.append(result)
# Took 10 seconds for 100 images
```

Question to AI:

"This code processes 100 images in 10 seconds. How can I optimize it for faster inference?"

AI Suggestion:

```
# Batch processing
batch_size = 16
for i in range(0, len(image_paths), batch_size):
    batch = image_paths[i:i+batch_size]
    results.extend(model(batch))
# Now takes 3 seconds for 100 images
````
```

### What I Learned:

- Batch processing is much more efficient
- GPU utilization improves with batches
- Trade-off between batch size and memory
- How to profile code for bottlenecks

### How I Applied It:

- Implemented batch inference in our demo
- Achieved >20 FPS target
- Added batch\_size as configurable parameter
- Documented performance improvements in results

## ✓ Summary Statistics

**Total Major AI Interactions:** 5 documented (plus ~15 minor queries)

### AI Tools Used:

- ChatGPT-4: 3 interactions (learning, optimization)
- Claude: 1 interaction (debugging)
- GitHub Copilot: 1 interaction (boilerplate code)

### Breakdown by Purpose:

- Learning CV concepts: 40%

- Debugging: 20%
- Code generation (non-CV): 20%
- Optimization: 20%

**Code Attribution:**

- CV model code: 90% written by us, 10% AI-assisted
- Data processing: 80% us, 20% AI-assisted
- Visualization/utilities: 50% us, 50% AI-assisted
- Training pipeline: 95% us, 5% AI-assisted

**Key Takeaway:** We used AI as a **learning accelerator** and **productivity tool**, not as a replacement for understanding. Every piece of AI-generated code was reviewed, understood, and often modified. We can explain all aspects of our project.

---

## Reflection

**What Worked Well:**

- Using AI to quickly understand complex concepts (YOLO architecture)
- Getting immediate help with debugging (saved hours)
- Generating boilerplate for non-CV tasks (visualization, utilities)

**What We Were Careful About:**

- Never asked AI to build entire project
- Always modified AI code to understand it
- Focused AI usage on learning, not avoiding learning
- Documented everything honestly

**Would We Do Differently:**

- Could have experimented more on our own before asking AI
- Sometimes relied on AI when documentation would have been better
- Should have explored alternative solutions more before accepting AI's first answer

---

**This log demonstrates responsible AI usage focused on learning and growth. 🎓**



## 6. LICENSE

MIT License

Copyright (c) 2025 [Your Name]

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

---

Note: This project uses the following open-source libraries:

- Ultralytics YOLOv8 (AGPL-3.0 License)
- PyTorch (BSD License)
- OpenCV (Apache 2.0 License)

For academic use only as part of ITAI 1378 coursework.

---



## 7. data/.gitkeep

```
This file keeps the directory in git
Delete this file once you add actual data
```



## 8. models/README.md

# Models Directory

## Structure

- `pretrained/`: Pre-trained models downloaded from public sources
- `trained/`: Our trained/fine-tuned models

# Pretrained Models

## YOLOv8n

- **File:** `yolo8n.pt`
- **Source:** Ultralytics official
- **Size:** 6.2 MB
- **Download:**

# Auto-downloads on first use, or manual:

```
wget https://github.com/ultralytics/assets/releases/download/v0.0.0/yolo8n.pt
```

## Trained Models

### Best Model (Final)

- **File:** `best.pt`
- **Training Date:** November 15, 2025
- **Metrics:**
  - mAP50: 0.873
  - mAP50-95: 0.651
  - Precision: 0.891
  - Recall: 0.847
- **Training Config:**
  - Epochs: 100 (early stopping at 87)
  - Image Size: 640
  - Batch Size: 16
  - Dataset: Custom warehouse + SKU-110K
- **Download:** [Google Drive Link] (for team members)

## Checkpoint Models

Saved every 10 epochs in case we need to revert.

## Model Weights (Too Large for Git)

Download trained models from:

- Google Drive: [LINK]
- Or retrain using:

```
python src/train.py --config config.yaml
```

## Usage Instructions

1. **Copy these templates into your repository**
2. **Fill in the placeholders** with your actual information
3. **Commit to GitHub**
4. **Update as you progress**

## Quick Setup Commands

```
Create repository structure
mkdir -p data/{raw,processed,sample} models/{pretrained,trained} notebooks src results/

Create .gitkeep files
touch data/raw/.gitkeep data/processed/.gitkeep models/trained/.gitkeep

Create key files
touch README.md requirements.txt .gitignore LICENSE
touch data/README.md models/README.md docs/AI_usage_log.md

Initialize git
git init
git add .
git commit -m "Initial project structure"
```

**These templates are your starting point. Customize them for your project! 🚀**